

Two core-Perl tools move a legacy model into SysML v2 text under Git: `bin/xmi2sysml.pl` reads a Cameo/MagicDraw XMI export (one `.sysml` file per top-level package); `bin/reqif2sysml.pl` reads a DOORS module as ReqIF or CSV (requirements with their `doorsId`, and the trace behind metrics C, F, G, I). Offline, Git for Windows only. They give a *starting point*: whatever cannot be mapped becomes a `// TODO` line with its `xmi:id`, never dropped, and a person reviews each segment before it is committed.

The daily loop

1. **Export** the model from Cameo and the requirements from DOORS into `exports/`, dated, outside the model repo.
2. **Read the report before converting:**

```
perl bin/xmi2sysml.pl --report exports/2026-10-05-legacy.xmi
perl bin/reqif2sysml.pl --report exports/2026-10-05-system.reqif
```

Check three things: the *unmapped* list, *stereotypes not converted*, *type references not resolved*. If one matters, fix the export first.
3. **Convert into a branch** of the model repo:


```
git switch -c port/structure
perl ../agile/bin/xmi2sysml.pl --out model --check \
  ../exports/2026-10-05-legacy.xmi
perl ../agile/bin/reqif2sysml.pl --out model \
  --layout "_reqs/{pkg}.sysml" --check ../exports/...reqif
```
4. **Review in Git:** `git status`, `git diff --stat`, then the diff segment by segment.
5. **Work the TODOs in Vim:** `:vimgrep /TODO/ model/**/*.sysml` then `:copen`; `:cnext`/`:cprev`. Each line names the v1 type and `xmi:id` to find it in Cameo.
6. **Lint:** `perl tools/sysml/model.pl lint` at 0 errors / 0 warnings (Vim: `\ml`).
7. **Commit one segment at a time:** `git add -p model/structure`; `message` `port: structure from legacy.xmi 2026-10-05`; `merge` through the gate.
8. **Collect the metrics:** `perl bin/status-metrics.pl collect ...`

Keep hand edits safe

The output is deterministic: the same export gives the same bytes. A re-run also *overwrites* your hand edits, so:

- Once a segment is ported and hand-fixed, **stop regenerating it**. Convert into `--out ../scratch` and copy over only the segments still being ported.
- Use `--report` to count what remains; its *ported vs remaining* line uses metric A's type filter.

xmi2sysml options

<code>--report</code>	mapping summary: in by <code>xmi:type</code> , ported vs remaining, the TODO list
<code>--out DIR</code>	write the files, layout <code>{seg}/{pkg}.sysml</code>
<code>--layout P</code>	another layout, e.g. <code>"{pkg}.sysml"</code>
<code>--stdout</code>	all files to stdout: <code>... vim -</code>
<code>--req-style</code>	usage (default, what the metrics count) or <code>def</code>
<code>--name PKG</code>	package for elements outside any package
<code>--check</code>	also run the SysML v2 grammar checker

reqif2sysml options

<code>--report</code>	requirements, verification methods, headings, links
<code>--model DIR</code>	with <code>--report</code> : trace C, F, G, I against the v2 text, with the ids
<code>--id-prefix P</code>	put back the module prefix DOORS drops (SYS-)
<code>--id-attr N</code>	the id is in another attribute
<code>--vm-attr N</code>	the verification-method attribute
<code>--attr N</code>	copy a DOORS attribute onto each requirement
<code>--csv, --col</code>	CSV input; <code>--col id="ID" --col text="Requirement"</code>

Exit codes (both): 0 ok, 1 the self-check or `--check` found a problem, 2 bad usage or unreadable file. Bad XML stops with `file:line: message`.

Becomes a TODO (finish by hand)

- Diagrams and layout (redraw v2 views from the text).
- State machines, interactions, operations; decision, merge, fork, join nodes, object flows, pins.
- OCL and opaque constraints (the expression is kept); constraint blocks (empty constraint `def`); parametrics beyond plain binding.
- Stereotypes outside SysML 1.x (company profiles, UAF, MARTE).
- Units: specialize the ISQ quantity by hand. DOORS images become `[image]`; tables flatten to `a | b`.

Git for the port

- One branch per segment: `git switch -c port/<segment>; merge` through the gate (lint 0/0, then review).
- Stage by hunk, `git add -p`, so a re-run cannot slip unreviewed changes in.
- Commit the export's date in the message; tag a baseline every four weeks (`git tag -a baseline-N`).
- Undo a bad conversion before commit: `git restore model/<segment>`.

What maps to what

SysML v1 (XMI)	SysML v2 text
Package (top level)	package (one file)
Block, Class, Component, Actor	part def; generalization :>
InterfaceBlock (or port-only Class)	port def; FlowProperty in/out item
Part property (composite)	part n : T[m..n];
Reference property; value property	ref part; attribute (with default)
ValueType, DataType; Enumeration	attribute def (unit as comment); enum def
Signal	item def
Port, Proxy, Full, Flow port	port n : [~]PortDef;
Connector; BindingConnector	connect a.p to b.q; bind a = b;
Activity	action def; first a then b;
Requirement (and Cameo kinds)	requirement { doorsId, title, verificationMethod }
Satisfy	satisfy R by X;
Verify, TestCase	verification def { objective { verify R; } }
DeriveReq	#derivation connection
Trace, Refine, Copy, Dependency; Allocate	dependency from A to B; allocate A to B;
Comment	doc /* ... */

Requirements are named from their id (CD-101 → CD_101) by both tools, so links written by one resolve against the other. Names that are not plain identifiers or are v2 reserved words are quoted: 'serial number'.

A SysML v2 sample (parses clean)

```
package Sample {
  private import ScalarValues::*;
  port def TrackPort { out attribute track : String; }
  part def Seeker { port out1 : TrackPort; }
  part def Guidance { port in1 : ~TrackPort; }
  part def KillVehicle {
    attribute massKg : Real = 25.0;
    part seeker : Seeker;
    part guidance : Guidance;
    connect seeker.out1 to guidance.in1;
  }
  requirement sysR101 {
    attribute doorsId = "SYS-101";
    doc /* Track the threat. */
  }
  satisfy sysR101 by KillVehicle;
  verification def FlightTest { objective { verify sysR101; } }
}
```

Exporting from Cameo / MagicDraw

- **File > Export To > Eclipse UML2 (v5.x) XMI File** (19.0+, 2021x–2024x). Older MagicDraw: *UML 2.1 XMI File*. Not the native .mdzip.
- **Include stereotype applications**, or Blocks come out as plain Classes and requirements lose id and text.
- **Used projects**: export libraries (e.g. SI value types) with the model, or fix those types by hand. Primitives map to ScalarValues::*.
- Several files may come out: give the converter the one with <uml:Model>. Diagrams are counted and skipped.

Exporting from DOORS

- **ReqIF** (preferred): DOORS 9 *File > Export > ReqIF*; DOORS Next *Export > ReqIF*. A .reqifz is read directly.
- Include **Object Heading, Object Text, the absolute number** and the verification-method attribute; add the prefix back with --id-prefix SYS-.
- **CSV fallback**: Object Identifier, Object Level, Object Heading, Object Text, Verification Method (found by name; others with --col).
- Heading only → a package (section); text → a requirement; neither → a counted TODO.

Checking the output

- Every run self-checks: balanced braces, strings and comments; every type, target, connect end and import resolves (by name, not scope).
- --check adds the grammar check: perl tools/sysml/sysml.pl check FILE... (Vim: \mx).
- perl tools/sysml/model.pl lint and check (outline rule, level tags) on the whole model (Vim: \ml, \mk).
- Not a full semantic validator: open a segment in a SysML v2 tool before you baseline it, when you can.

How the port feeds A–Z

- **A**: status-metrics.pl init --xmi freezes the denominator with uml:(Class|Port|Property|Connector|Activity); the report's *ported* vs *remaining* counts the same set.
- **C, F, G, I**: reqif2sysml.pl --report --model model export.reqif: linked, orphans, unverified, trace %, with the ids for the owners.
- Commit the default --req-style usage: it is the form the collector counts.

Full guide: docs/SYSML-PORT.html; help: perl agile.pl help xmi2sysml; the worked model: examples/halberd/.